

Anatolie Ernu

IT & Security Solutions

<https://www.ernu.eu>

TECHNICAL GUIDE

WordPress Installation & Hardening

Debian 13 Trixie · Nginx · MariaDB 10.11 · PHP 8.3-FPM

Version: 1.0 | 2026 | Enterprise Edition

Category	Details
Operating System	Debian 13 (Trixie) – minimal install
Web Server	Nginx 1.24+ (stable)
Database	MariaDB 10.11 LTS
PHP Runtime	PHP 8.3-FPM
Firewall	UFW + Fail2Ban + ModSecurity (OWASP CRS)
TLS/SSL	Certbot (Let's Encrypt) or custom certificate
Auto-update	unattended-upgrades
Monitoring	Wazuh Agent + auditd

TABLE OF CONTENTS

1. SYSTEM PREPARATION – DEBIAN 13

1.1 Minimal Debian 13 Installation

A minimal netinstall image is recommended to obtain a clean system without unnecessary packages.

|  Download *debian-13.x.x-amd64-netinst.iso* from <https://www.debian.org/distrib/netinst>

Recommended installation options:

- Partitioning: separate / from /var and /home
- Software selection: select ONLY 'SSH server' and 'standard system utilities'
- DO NOT install Desktop Environment, print server, or other services
- Create a non-root user for administration

1.2 Initial Post-Installation Configuration

1.2.1 Complete System Update

```
# Update package lists and upgrade system
apt update && apt upgrade -y
```

```
# Install essential packages
apt install -y curl wget gnupg2 ca-certificates lsb-release \
  apt-transport-https software-properties-common unzip \
  vim htop net-tools ufw fail2ban auditd git
```

1.2.2 Hostname and Timezone Configuration

```
# Set server hostname
hostnamectl set-hostname web01.ernu.sec
```

```
# Set timezone to Moldova
timedatectl set-timezone Europe/Chisinau
```

```
# Enable NTP synchronization
timedatectl set-ntp true
timedatectl status
```

1.2.3 SSH Hardening

|  **Change the SSH port BEFORE enabling the firewall to avoid locking yourself out!**

```
# Edit SSH configuration
nano /etc/ssh/sshd_config
```

Mandatory parameters in /etc/ssh/sshd_config:

```
Port 2222                # Change default port
Protocol 2               # Force SSH v2
PermitRootLogin no       # Deny root login
PasswordAuthentication no # Key-based auth only
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
MaxAuthTries 3           # Max auth attempts
MaxSessions 5
LoginGraceTime 30
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no
UseDNS no
Banner /etc/ssh/banner.txt
AllowUsers adminuser      # Replace with your user
# Create warning banner
echo 'Authorized access only. All sessions are monitored.' > /etc/ssh/banner.txt
```

```
# Restart SSH
systemctl restart sshd
```

✓ SSH is now configured to accept only public keys on a non-standard port

1.2.4 UFW Firewall Configuration

```
# Reset UFW rules
ufw --force reset
```

```
# Default policy: deny all incoming
ufw default deny incoming
ufw default allow outgoing
```

```
# Allow SSH on the new port
ufw allow 2222/tcp comment 'SSH'
```

```
# Allow HTTP and HTTPS
ufw allow 80/tcp comment 'HTTP'
ufw allow 443/tcp comment 'HTTPS'
```

```
# Enable UFW
ufw --force enable
ufw status verbose
```

1.2.5 Kernel Configuration – sysctl Hardening Parameters

```
# Create sysctl hardening configuration file
cat > /etc/sysctl.d/99-hardening.conf << 'EOF'
# IP Spoofing Protection
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
```

```
# Disable IP packet forwarding
net.ipv4.ip_forward = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0
```

```
# Ignore ICMP redirects
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_redirects = 0
```

```
# SYN Flood Protection
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2
net.ipv4.tcp_syn_retries = 5
```

```
# Disable ping (optional)
net.ipv4.icmp_echo_ignore_all = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1
```

```
# Buffer overflow protection
kernel.randomize_va_space = 2
kernel.exec-shield = 1
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
EOF
```

```
# Apply settings
sysctl --system
```

1.2.6 Automatic Security Updates

```
apt install -y unattended-upgrades apt-listchanges
```

```
# Configure automatic updates
cat > /etc/apt/apt.conf.d/50unattended-upgrades << 'EOF'
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}ESMApps:${distro_codename}-apps-security";
};
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";
Unattended-Upgrade::Mail "admin@ernu.sec";
EOF
```

```
dpkg-reconfigure -pmedium unattended-upgrades
```

2. MARIADB 10.11 INSTALLATION AND CONFIGURATION

2.1 MariaDB Installation

```
# Add the official MariaDB repository
curl -LsS https://downloads.mariadb.com/MariaDB/mariadb_repo_setup \
  | bash -s -- --mariadb-server-version=10.11

# Install MariaDB
apt update
apt install -y mariadb-server mariadb-client

# Enable and start the service
systemctl enable --now mariadb
systemctl status mariadb
```

2.2 MariaDB Security Hardening (mysql_secure_installation)

 Run the interactive security script immediately after installation

mysql_secure_installation

Answer the prompts as follows:

- Enter current password for root: [Enter] (blank on first run)
- Switch to unix_socket authentication: Y
- Change the root password: Y (set a strong password)
- Remove anonymous users: Y
- Disallow root login remotely: Y
- Remove test database: Y
- Reload privilege tables: Y

2.3 Create WordPress Database and User

 Use passwords of at least 24 characters with uppercase/lowercase, digits, and special characters!

```
# Connect to MariaDB as root
mysql -u root -p
```

```
-- Replace 'STRONG_DB_PASSWORD' with an actual password
CREATE DATABASE wp_site1 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp_user1'@'localhost' IDENTIFIED BY 'STRONG_DB_PASSWORD';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
  ON wp_site1.* TO 'wp_user1'@'localhost';
FLUSH PRIVILEGES;
```

```
-- Verify the created user
SELECT User, Host, plugin FROM mysql.user WHERE User='wp_user1';
SHOW GRANTS FOR 'wp_user1'@'localhost';
EXIT;
```

2.4 MariaDB Configuration Hardening

```
# Edit the main configuration file
nano /etc/mysql/mariadb.conf.d/50-server.cnf

Add/modify these parameters under the [mysqld] section:

[mysqld]
# Disable network access (WordPress is on the same server)
bind-address = 127.0.0.1
skip-networking = 0

# Security
```

```
local-infile = 0
symbolic-links = 0
skip-show-database

# Performance and stability
max_connections = 100
max_allowed_packet = 64M
tmp_table_size = 32M
max_heap_table_size = 32M

# Logging for audit
general_log = 0
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow-query.log
long_query_time = 2

# Character set
character-set-server = utf8mb4
collation-server = utf8mb4_unicode_ci

# Restart MariaDB
systemctl restart mariadb
systemctl status mariadb
```

✓ MariaDB is configured to accept connections only from localhost

3. PHP 8.3-FPM INSTALLATION AND CONFIGURATION

3.1 Add PHP Repository and Install

```
# Add the Sury repository (updated PHP packages)
curl -sSLo /usr/share/keyrings/deb.sury.org-php.gpg \
  https://packages.sury.org/php/apt.gpg
echo "deb [signed-by=/usr/share/keyrings/deb.sury.org-php.gpg] \
  https://packages.sury.org/php/ $(lsb_release -sc) main" \
  > /etc/apt/sources.list.d/php.list
apt update

# Install PHP 8.3 and required extensions for WordPress
apt install -y php8.3-fpm php8.3-mysql php8.3-xml php8.3-gd \
  php8.3-mbstring php8.3-curl php8.3-zip php8.3-intl \
  php8.3-bcmath php8.3-soap php8.3-imagick php8.3-redis \
  php8.3-opcache

# Verify version
php8.3 --version
php8.3-fpm --version
```

3.2 php.ini Hardening

```
# Backup original configuration
cp /etc/php/8.3/fpm/php.ini /etc/php/8.3/fpm/php.ini.bak

# Edit PHP-FPM configuration
nano /etc/php/8.3/fpm/php.ini

Mandatory security parameters in php.ini:

; GENERAL SECURITY
expose_php = Off           ; Hide PHP version from headers
display_errors = Off       ; Do not display errors in browser
display_startup_errors = Off
log_errors = On            ; Log errors to file
error_log = /var/log/php8.3-fpm/php_errors.log
error_reporting = E_ALL & ~E_DEPRECATED & ~E_STRICT

; DANGEROUS FUNCTIONS
disable_functions = exec,passthru,shell_exec,system,proc_open,\
  popen,curl_exec,curl_multi_exec,parse_ini_file,show_source,\
  phpinfo,pcntl_alarm,pcntl_fork,pcntl_waitpid,pcntl_wait,\
  pcntl_wifexited,pcntl_wifstopped,pcntl_wifsignaled,\
  pcntl_wifcontinued,pcntl_wexitstatus,pcntl_wtermsig,\
  pcntl_wstopsig,pcntl_signal,pcntl_signal_get_handler,\
  pcntl_signal_dispatch,pcntl_get_last_error,pcntl_strerror,\
  pcntl_sigprocmask,pcntl_sigwaitinfo,pcntl_sigtimedwait,\
  pcntl_exec,pcntl_getpriority,pcntl_setpriority,pcntl_async_signals

; UPLOAD AND MEMORY
file_uploads = On
upload_max_filesize = 64M
post_max_size = 64M
max_execution_time = 300
max_input_time = 300
memory_limit = 256M
max_input_vars = 3000

; SESSIONS
session.cookie_httponly = On
session.cookie_secure = On
session.cookie_samesite = Strict
session.use_strict_mode = On
session.use_only_cookies = On
```

```
session.gc_maxlifetime = 1440

; OPENBASEDIR - restrict filesystem access
; open_basedir = /var/www/site1:/tmp:/usr/share/php
; (uncomment and adapt to your directory structure)

; OPCACHE
opcache.enable = 1
opcache.memory_consumption = 128
opcache.interned_strings_buffer = 8
opcache.max_accelerated_files = 4000
opcache.revalidate_freq = 60
opcache.fast_shutdown = 1
opcache.enable_cli = 0
```

3.3 PHP-FPM Pool Configuration

Each WordPress site must run under its own dedicated FPM pool with a dedicated user:

```
# Create a dedicated system user for the site
useradd -r -s /usr/sbin/nologin -d /var/www/site1 wp_site1

# Create dedicated FPM pool for site1
cat > /etc/php/8.3/fpm/pool.d/site1.conf << 'EOF'
[site1]
user = wp_site1
group = wp_site1
listen = /run/php/php8.3-site1.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660

; Process management type
pm = dynamic
pm.max_children = 20
pm.start_servers = 5
pm.min_spare_servers = 3
pm.max_spare_servers = 10
pm.max_requests = 500

; Security
security.limit_extensions = .php
php_admin_value[disable_functions] = exec,passthru,shell_exec,system
php_admin_flag[allow_url_fopen] = off
php_admin_flag[allow_url_include] = off
php_admin_value[open_basedir] = /var/www/site1:/tmp
php_admin_value[error_log] = /var/log/php8.3-fpm/site1-error.log
php_admin_flag[log_errors] = on

; Environment variables
env[HOSTNAME] = $HOSTNAME
env[TMP] = /tmp
env[TMPDIR] = /tmp
env[TEMP] = /tmp
EOF

# Restart PHP-FPM
systemctl restart php8.3-fpm
systemctl status php8.3-fpm
```

4. NGINX INSTALLATION AND HARDENING

4.1 Nginx Installation

```
# Add the official Nginx repository
curl -fsSL https://nginx.org/keys/nginx_signing.key \
  | gpg --dearmor -o /usr/share/keyrings/nginx.gpg
echo "deb [signed-by=/usr/share/keyrings/nginx.gpg] \
  https://nginx.org/packages/debian $(lsb_release -sc) nginx" \
  > /etc/apt/sources.list.d/nginx.list
apt update
apt install -y nginx
systemctl enable --now nginx
nginx -v
```

4.2 Global Nginx Hardening Configuration

Edit the main configuration file:

```
# Backup original configuration
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

cat > /etc/nginx/nginx.conf << 'EOF'
user www-data;
worker_processes auto;
worker_rlimit_nofile 65535;
pid /run/nginx.pid;

events {
    worker_connections 1024;
    multi_accept on;
    use epoll;
}

http {
    # Base
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    charset utf-8;

    # Hide Nginx version
    server_tokens off;

    # Performance
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;

    # Request limits
    client_body_buffer_size 16K;
    client_header_buffer_size 1k;
    client_max_body_size 64m;
    large_client_header_buffers 4 16k;
    client_body_timeout 12;
    client_header_timeout 12;
    send_timeout 10;

    # Rate Limiting - DDoS/brute-force protection
    limit_req_zone $binary_remote_addr zone=general:10m rate=30r/m;
    limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
    limit_req_zone $binary_remote_addr zone=xmlrpc:10m rate=2r/m;
    limit_conn_zone $binary_remote_addr zone=addr:10m;
```

```
# Gzip
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_comp_level 6;
gzip_types text/plain text/css text/xml text/javascript
        application/json application/javascript application/xml
        application/rss+xml font/ttf image/svg+xml;

# Logging
log_format main '$remote_addr - $remote_user [$time_local]'
    '"$request" $status $body_bytes_sent'
    '"$http_referer" "$http_user_agent"'
    '"$http_x_forwarded_for" rt=$request_time';

access_log /var/log/nginx/access.log main;
error_log /var/log/nginx/error.log warn;

# Include virtual hosts
include /etc/nginx/conf.d/*.conf;
include /etc/nginx/sites-enabled/*;
}
EOF
```

4.3 WordPress Virtual Host Configuration with TLS

⚠️ Replace wp01.ernu.sec with your actual domain before applying!

```
# Install Certbot for Let's Encrypt
apt install -y certbot python3-certbot-nginx

# Obtain SSL certificate (run AFTER Nginx starts on port 80)
certbot --nginx -d wp01.ernu.sec \
    --email admin@ernu.sec --agree-tos --no-eff-email

Create the Virtual Host file for WordPress:

cat > /etc/nginx/sites-available/site1.conf << 'NGINXEOF'
# HTTP -> HTTPS redirect
server {
    listen 80;
    listen [::]:80;
    server_name wp01.ernu.sec;
    return 301 https://$host$request_uri;
}

# Main HTTPS server
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name wp01.ernu.sec;
    root /var/www/site1;
    index index.php;

    # TLS/SSL
    ssl_certificate /etc/letsencrypt/live/wp01.ernu.sec/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/wp01.ernu.sec/privkey.pem;
    ssl_session_timeout 1d;
    ssl_session_cache shared:MozSSL:10m;
    ssl_session_tickets off;

    # Modern TLS (Mozilla Intermediate)
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
        ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
        ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:
```

```
DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384;
ssl_prefer_server_ciphers off;

# HSTS - force HTTPS for 2 years
add_header Strict-Transport-Security 'max-age=63072000; includeSubDomains; preload'
always;

# Security Headers
add_header X-Frame-Options 'SAMEORIGIN' always;
add_header X-Content-Type-Options 'nosniff' always;
add_header X-XSS-Protection '1; mode=block' always;
add_header Referrer-Policy 'strict-origin-when-cross-origin' always;
add_header Permissions-Policy 'camera=(),microphone=(),geolocation=()' always;

# Rate Limiting
limit_conn addr 20;

# WordPress - Main rules
location / {
    limit_req zone=general burst=20 nodelay;
    try_files $uri $uri/ /index.php?$args;
}

# wp-login.php protection
location = /wp-login.php {
    limit_req zone=login burst=3 nodelay;
    fastcgi_pass unix:/run/php/php8.3-site1.sock;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    include fastcgi_params;
}

# Block xmlrpc.php
location = /xmlrpc.php {
    limit_req zone=xmlrpc burst=1 nodelay;
    deny all;
    return 403;
}

# PHP processing
location ~ /\.php$ {
    try_files $uri =404;
    fastcgi_split_path_info ^(.+\.php)(/.+)$;
    fastcgi_pass unix:/run/php/php8.3-site1.sock;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    include fastcgi_params;
    fastcgi_intercept_errors on;
    fastcgi_buffer_size 128k;
    fastcgi_buffers 256 16k;
    fastcgi_busy_buffers_size 256k;
    fastcgi_read_timeout 300;
}

# Block access to sensitive files
location ~ /\. { deny all; }
location ~ /wp-config\.php { deny all; }
location ~* \.(?:log|bak|sql|tar|gz|zip|env|ini|conf)$ { deny all; }
location ~* /(?:readme|license|changelog|readme\.html)$ { deny all; }

# Block PHP execution in uploads
location ~* /wp-content/uploads/.*\.php$ {
    deny all;
    return 403;
}
```

```
# Static files - aggressive caching
location ~* \.(?:css|js|jpg|jpeg|gif|png|ico|svg|woff|woff2|ttf|eot)$ {
    expires 30d;
    add_header Cache-Control 'public, no-transform';
    access_log off;
}

# Per-site logs
access_log /var/log/nginx/site1-access.log main;
error_log /var/log/nginx/site1-error.log warn;
}
NGINXEOF
```

```
# Enable site
ln -s /etc/nginx/sites-available/site1.conf /etc/nginx/sites-enabled/
nginx -t
systemctl reload nginx
```

✓ Virtual Host configured with TLS 1.2/1.3, security headers, and rate limiting

5. WORDPRESS INSTALLATION AND HARDENING

5.1 Download and Install WordPress

```
# Create site directory
mkdir -p /var/www/site1

# Download latest WordPress version
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz

# Copy files
cp -r wordpress/* /var/www/site1/

# Set correct ownership
chown -R wp_site1:wp_site1 /var/www/site1/

# Set correct permissions
find /var/www/site1 -type d -exec chmod 755 {} \;
find /var/www/site1 -type f -exec chmod 644 {} \;

# Special permissions for writable directories
chmod 755 /var/www/site1/wp-content
chmod 755 /var/www/site1/wp-content/uploads
chmod 755 /var/www/site1/wp-content/themes
chmod 755 /var/www/site1/wp-content/plugins
```

5.2 Hardened wp-config.php Configuration

```
# Copy the configuration template
cp /var/www/site1/wp-config-sample.php /var/www/site1/wp-config.php

# Generate unique security keys (insert output into wp-config.php)
curl -s https://api.wordpress.org/secret-key/1.1/salt/
Edit /var/www/site1/wp-config.php with the parameters below:

<?php
// Database
define('DB_NAME', 'wp_site1');
define('DB_USER', 'wp_user1');
define('DB_PASSWORD', 'STRONG_DB_PASSWORD');
define('DB_HOST', '127.0.0.1');
define('DB_CHARSET', 'utf8mb4');
define('DB_COLLATE', 'utf8mb4_unicode_ci');

// Custom table prefix (do NOT use wp_)
$table_prefix = 'x7r2_'; // generate randomly

// Authentication keys (insert output from above)
define('AUTH_KEY', 'unique-key-1');
define('SECURE_AUTH_KEY', 'unique-key-2');
define('LOGGED_IN_KEY', 'unique-key-3');
define('NONCE_KEY', 'unique-key-4');
define('AUTH_SALT', 'unique-key-5');
define('SECURE_AUTH_SALT', 'unique-key-6');
define('LOGGED_IN_SALT', 'unique-key-7');
define('NONCE_SALT', 'unique-key-8');

// Security
define('DISALLOW_FILE_EDIT', true); // Disable admin file editor
define('DISALLOW_FILE_MODS', false); // Allow plugin/theme updates
define('FORCE_SSL_ADMIN', true); // Admin HTTPS only
define('WP_DEBUG', false); // NOT in production!
```

```
define('WP_DEBUG_LOG', false);
define('WP_DEBUG_DISPLAY', false);
define('SCRIPT_DEBUG', false);

// Limit post revisions
define('WP_POST_REVISIONS', 5);

// Auto-update
define('WP_AUTO_UPDATE_CORE', true);

// Directory protection
define('WP_CONTENT_DIR', '/var/www/site1/wp-content');
define('WP_CONTENT_URL', 'https://wp01.ernu.sec/wp-content');

// ABSPATH
if (!defined('ABSPATH')) define('ABSPATH', __DIR__ . '/');
require_once ABSPATH . 'wp-settings.php';
# Protect wp-config.php with restrictive permissions
chmod 640 /var/www/site1/wp-config.php
chown wp_site1:www-data /var/www/site1/wp-config.php
```

5.3 .htaccess Equivalent Protection via Nginx

 The Nginx security rules from Chapter 4 already cover the protection equivalent to WordPress .htaccess. No additional configuration is required.

5.4 Finalize WordPress Installation

```
# Verify all services are running
systemctl status nginx
systemctl status php8.3-fpm
systemctl status mariadb
```

```
# Access the site URL in browser to run the install wizard
# https://wp01.ernu.sec/wp-admin/install.php
```

 **During WordPress installation: choose an admin username that is NOT 'admin'! Use a randomly generated password of at least 20 characters.**

6. FAIL2BAN CONFIGURATION

6.1 Installation and Basic Configuration

```
# Install Fail2Ban
apt install -y fail2ban

# Copy local configuration (do not modify jail.conf directly!)
cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

6.2 Custom Jail Configuration

```
cat > /etc/fail2ban/jail.d/wordpress.local << 'EOF'
[DEFAULT]
bantime = 3600           ; ban for 1 hour
findtime = 600           ; 10-minute window
maxretry = 5             ; 5 failed attempts
banaction = ufw

[sshd]
enabled = true
port = 2222
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 86400          ; 24-hour ban for SSH

[nginx-http-auth]
enabled = true
port = http,https
filter = nginx-http-auth
logpath = /var/log/nginx/error.log

[nginx-limit-req]
enabled = true
port = http,https
filter = nginx-limit-req
logpath = /var/log/nginx/error.log
maxretry = 10

[wordpress-auth]
enabled = true
port = http,https
filter = wordpress-auth
logpath = /var/log/nginx/site1-access.log
maxretry = 5
bantime = 7200
EOF
# Create custom filter for WordPress
cat > /etc/fail2ban/filter.d/wordpress-auth.conf << 'EOF'
[Definition]
failregex = ^<HOST> .* "POST /wp-login\.php
            ^<HOST> .* "POST /xmlrpc\.php

ignoreregex =
datepattern = {^LN-BEG}%d/%b/%Y:%H:%M:%S %Z
EOF

# Restart and verify Fail2Ban
systemctl enable --now fail2ban
systemctl status fail2ban
fail2ban-client status
```

✓ Fail2Ban protects SSH, Nginx, and wp-login.php against brute-force attacks

7. ADVANCED PROTECTION – ModSecurity + OWASP CRS

7.1 Compile ModSecurity with Nginx

 *ModSecurity v3 (libmodsecurity) works with Nginx through the dynamic module nginx-module-modsecurity*

```
# Install compilation dependencies
apt install -y libmodsecurity3 libmodsecurity-dev

# Install the Nginx-ModSecurity connector
apt install -y libnginx-mod-http-modsecurity

# Enable the module in Nginx
echo 'load_module /usr/lib/nginx/modules/ngx_http_modsecurity_module.so;' \
> /etc/nginx/modules-enabled/50-mod-http-modsecurity.conf
```

7.2 Download OWASP Core Rule Set

```
# Download OWASP CRS
cd /tmp
wget https://github.com/coreruleset/coreruleset/archive/refs/tags/v3.3.5.tar.gz
tar -xzf v3.3.5.tar.gz
mv coreruleset-3.3.5 /etc/nginx/modsec-crs

# Configure CRS
cp /etc/nginx/modsec-crs/crs-setup.conf.example \
/etc/nginx/modsec-crs/crs-setup.conf

# Copy ModSecurity configuration
cp /etc/modsecurity/modsecurity.conf-recommended \
/etc/modsecurity/modsecurity.conf

# Enable ModSecurity (change from DetectionOnly to On)
sed -i 's/SecRuleEngine DetectionOnly/SecRuleEngine On/' \
/etc/modsecurity/modsecurity.conf

# Create main ModSecurity file for Nginx
mkdir -p /etc/nginx/modsec
cat > /etc/nginx/modsec/main.conf << 'EOF'
Include /etc/modsecurity/modsecurity.conf
Include /etc/nginx/modsec-crs/crs-setup.conf
Include /etc/nginx/modsec-crs/rules/*.conf
EOF

# Add to the server block in Virtual Host:
# modsecurity on;
# modsecurity_rules_file /etc/nginx/modsec/main.conf;

# Test and reload Nginx
nginx -t && systemctl reload nginx
```

8. FILE PERMISSIONS AND SYSTEM

8.1 Permission Verification and Setting Script

```
#!/bin/bash
# Script: fix-wp-permissions.sh
# Run after each installation or major update

WP_PATH="/var/www/site1"
WP_USER="wp_site1"
WEB_GROUP="www-data"

echo '=== Setting WordPress Permissions ==='

# Ownership
chown -R ${WP_USER}:${WEB_GROUP} ${WP_PATH}

# Directories: 755 (rwxr-xr-x)
find ${WP_PATH} -type d -exec chmod 755 {} \;

# Files: 644 (rw-r--r--)
find ${WP_PATH} -type f -exec chmod 644 {} \;

# wp-config.php: 640 (rw-r-----)
chmod 640 ${WP_PATH}/wp-config.php

# .htaccess: 444 (r--r--r--) - read-only
[ -f ${WP_PATH}/.htaccess ] && chmod 444 ${WP_PATH}/.htaccess

echo '=== Verifying wp-config.php permissions ==='
ls -la ${WP_PATH}/wp-config.php
echo '=== Done! ==='

# Save and run the script
chmod +x /usr/local/bin/fix-wp-permissions.sh
/usr/local/bin/fix-wp-permissions.sh
```

8.2 Directory Protection with Special Permissions

```
# Prevent direct access to PHP files in uploads
# (already configured in Nginx Virtual Host)

# Verify no PHP files exist in uploads
find /var/www/site1/wp-content/uploads -name '*.php' -exec rm {} \;

# Check for incorrect permissions
find /var/www/site1 -type f -perm /o+w -exec ls -la {} \;
```

9. AUTOMATED BACKUP

9.1 Complete Backup Script

```
#!/bin/bash
# Script: wp-backup.sh - WordPress + MariaDB Backup

SITE_NAME="site1"
WP_PATH="/var/www/site1"
DB_NAME="wp_site1"
DB_USER="wp_user1"
DB_PASS="STRONG_DB_PASSWORD"
BACKUP_DIR="/var/backups/wordpress"
RETENTION_DAYS=30
DATE=$(date +%Y%m%d_%H%M%S)

mkdir -p ${BACKUP_DIR}
echo "[$(date)] Starting backup ${SITE_NAME}"

# 1. Database backup
mysqldump -u${DB_USER} -p${DB_PASS} --single-transaction \
--quick --lock-tables=false ${DB_NAME} \
| gzip > ${BACKUP_DIR}/${SITE_NAME}_db_${DATE}.sql.gz

# 2. WordPress files backup
tar -czf ${BACKUP_DIR}/${SITE_NAME}_files_${DATE}.tar.gz \
--exclude='*.log' \
-C /var/www ${SITE_NAME}/

# 3. Delete backups older than RETENTION_DAYS
find ${BACKUP_DIR} -name "${SITE_NAME}_*.gz" \
-mtime +${RETENTION_DAYS} -delete

echo "[$(date)] Backup completed: ${BACKUP_DIR}"
ls -lh ${BACKUP_DIR}/${SITE_NAME}_*_${DATE}*

# Save script and set permissions
chmod 700 /usr/local/bin/wp-backup.sh

# Add to crontab (daily backup at 02:00)
crontab -e
# Add line:
0 2 * * * /usr/local/bin/wp-backup.sh >> /var/log/wp-backup.log 2>&1
```

10. MONITORING AND AUDIT

10.1 auditd Configuration

```
# Install auditd
apt install -y auditd audispd-plugins

# Add audit rules for WordPress
cat >> /etc/audit/rules.d/wordpress.rules << 'EOF'
# Monitor wp-config.php changes
-w /var/www/site1/wp-config.php -p wa -k wp_config_change

# Monitor uploads directory (file execution)
-w /var/www/site1/wp-content/uploads/ -p x -k wp_uploads_exec

# Monitor new PHP files in uploads
-w /var/www/site1/wp-content/uploads/ -p wa -k wp_uploads_write

# Monitor core WordPress file changes
-w /var/www/site1/wp-admin/ -p wa -k wp_admin_change
-w /var/www/site1/wp-includes/ -p wa -k wp_includes_change
EOF

# Apply rules
augenrules --load
systemctl restart auditd

# Verify active rules
auditctl -l
```

10.2 WordPress File Integrity Verification (WP-CLI)

```
# Install WP-CLI
curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar
chmod +x wp-cli.phar
mv wp-cli.phar /usr/local/bin/wp

# Verify WordPress core integrity
sudo -u wp_site1 wp --path=/var/www/site1 core verify-checksums

# Verify plugins
sudo -u wp_site1 wp --path=/var/www/site1 plugin verify-checksums --all

# List available updates
sudo -u wp_site1 wp --path=/var/www/site1 core check-update
sudo -u wp_site1 wp --path=/var/www/site1 plugin list --update=available

# Automated daily verification script
cat > /usr/local/bin/wp-integrity-check.sh << 'EOF'
#!/bin/bash
echo "[$(date)] WordPress integrity verification"
sudo -u wp_site1 wp --path=/var/www/site1 core verify-checksums 2>&1
sudo -u wp_site1 wp --path=/var/www/site1 plugin verify-checksums --all 2>&1
EOF
chmod 700 /usr/local/bin/wp-integrity-check.sh

# Add to crontab (daily check at 06:00)
# 0 6 * * * /usr/local/bin/wp-integrity-check.sh >> /var/log/wp-integrity.log 2>&1
```

11. FINAL VERIFICATION CHECKLIST

11.1 Quick System Verification

```
#!/bin/bash
# Quick post-installation security verification script
echo '=====
echo ' SERVER SECURITY VERIFICATION '
echo '=====

# Active services
echo ''
echo '--- Services ---'
systemctl is-active nginx php8.3-fpm mariadb fail2ban ufw auditd

# UFW status
echo ''
echo '--- UFW Firewall ---'
ufw status

# Fail2Ban jails
echo ''
echo '--- Fail2Ban ---'
fail2ban-client status

# TLS check
echo ''
echo '--- SSL Certificate ---'
openssl s_client -connect wp01.ernu.sec:443 </dev/null 2>&1 \
  | grep -E 'subject|issuer|Verify'

# wp-config.php permissions
echo ''
echo '--- wp-config.php Permissions ---'
stat -c '%a %U:%G %n' /var/www/site1/wp-config.php

# PHP files in uploads
echo ''
echo '--- PHP Files in uploads ---'
find /var/www/site1/wp-content/uploads -name '*.php' 2>/dev/null \
  && echo 'WARNING: PHP files found!' \
  || echo 'OK: no PHP files'

echo ''
echo '=====
echo ' VERIFICATION COMPLETE '
echo '=====
```

11.2 Security Summary Table

Component	Security Measure	Status
Debian OS	Automatic security updates active	✓
SSH	Non-standard port, no root login, key-only auth	✓
UFW Firewall	Default deny incoming, allow 2222/80/443 explicit	✓
Sysctl	SYN flood protection, IP spoofing, ASLR active	✓
MariaDB	Bind localhost, no remote root, strong passwords	✓
PHP 8.3-FPM	disable_functions, open_basedir, opcache, secure sessions	✓
Nginx	server_tokens off, rate limiting, security headers, TLS 1.2/1.3	✓

TLS/SSL	Certbot Let's Encrypt, HSTS, modern ciphers	✓
WordPress	DISALLOW_FILE_EDIT, random table prefix, FORCE_SSL_ADMIN	✓
Fail2Ban	SSH, Nginx, wp-login.php protection	✓
ModSecurity	OWASP CRS v3.3 active – WAF protection	✓
Permissions	wp-config.php 640, uploads without PHP exec	✓
Backup	Daily DB + file backup, 30-day retention	✓
Audit	auditd monitors wp-config.php, uploads, core files	✓
WP-CLI	Daily core + plugin integrity verification	✓

11.3 Recommended WordPress Security Plugins

Plugin	Function	Notes
Wordfence Security	Application firewall, malware scan, 2FA	Free version is sufficient
WP 2FA	Two-factor authentication for admin	Mandatory for admin account
UpdraftPlus	Additional cloud backup (S3, GDrive)	Off-site backup
WPS Hide Login	Changes login page URL	Prevents automated attacks
Disable XML-RPC	Completely disables xmlrpc.php	Alternative to Nginx blocking
Query Monitor	Performance debugging (dev/staging only)	Disable in production

12. SECOND WORDPRESS SITE (site2)

Repeat the steps below for the second site, replacing site1 with site2:

```
# 1. Database
mysql -u root -p -e "
    CREATE DATABASE wp_site2 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
    CREATE USER 'wp_user2'@'localhost' IDENTIFIED BY 'ANOTHER_STRONG_PASSWORD';
    GRANT SELECT,INSERT,UPDATE,DELETE,CREATE,DROP,INDEX,ALTER
        ON wp_site2.* TO 'wp_user2'@'localhost';
    FLUSH PRIVILEGES;
"

# 2. Dedicated system user
useradd -r -s /usr/sbin/nologin -d /var/www/site2 wp_site2

# 3. Dedicated PHP-FPM pool
cp /etc/php/8.3/fpm/pool.d/site1.conf \
    /etc/php/8.3/fpm/pool.d/site2.conf
sed -i 's/site1/site2/g; s/wp_site1/wp_site2/g' \
    /etc/php/8.3/fpm/pool.d/site2.conf
systemctl restart php8.3-fpm

# 4. Nginx Virtual Host
cp /etc/nginx/sites-available/site1.conf \
    /etc/nginx/sites-available/site2.conf
sed -i 's/site1/site2/g' /etc/nginx/sites-available/site2.conf
ln -s /etc/nginx/sites-available/site2.conf \
    /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx

# 5. SSL Certificate
certbot --nginx -d wp02.ernu.sec \
    --email admin@ernu.sec --agree-tos --no-eff-email

# 6. Install WordPress
mkdir -p /var/www/site2
cp -r /tmp/wordpress/* /var/www/site2/
chown -R wp_site2:www-data /var/www/site2/
find /var/www/site2 -type d -exec chmod 755 {} \;
find /var/www/site2 -type f -exec chmod 644 {} \;
```

✓ Each site runs in isolation: own user, own FPM pool, own database, own Virtual Host.

13. CLOUDFLARE + NGINX PROXY MANAGER – REAL IP IN LOGS

13.1 Traffic Flow Diagram

Traffic traverses three levels before reaching WordPress:

```
[Visitor]
| Real IP: 85.x.x.x
v
[Cloudflare CDN / WAF]
| Adds header: CF-Connecting-IP: 85.x.x.x
| Header: X-Forwarded-For: 85.x.x.x
| Source IP to NPM: 104.x.x.x (Cloudflare IP)
v
[Nginx Proxy Manager]
| Forwards: X-Real-IP, X-Forwarded-For
| Source IP to Nginx WP: 192.168.x.x (NPM internal IP)
v
[Nginx WordPress - your server]
| Without config: sees NPM IP, NOT the real IP!
| With correct config: sees 85.x.x.x (real IP)
v
[MariaDB + PHP-FPM]
```

⚠ Without the configuration below, Nginx logs and Fail2Ban see only the NPM IP (192.168.x.x) or Cloudflare IPs (104.x.x.x) – never the real attacker IP!

13.2 Step 1 – Configure Nginx Proxy Manager

13.2.1 Advanced Configuration per Proxy Host in NPM

In the NPM web interface: Edit Proxy Host → Advanced tab → Custom Nginx Configuration:

```
# NPM Advanced Config (per Proxy Host)
# Forward the real Cloudflare IP to backend Nginx WP
proxy_set_header X-Real-IP $http_cf_connecting_ip;
proxy_set_header X-Forwarded-For $http_cf_connecting_ip;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header Host $host;
```

13.2.2 Global NPM Configuration (optional – Docker)

If you have access to the NPM server, edit the global file /data/nginx/custom/http.conf:

```
# /data/nginx/custom/http.conf (on the NPM server)
# Cloudflare IPs declared as trusted sources
set_real_ip_from 103.21.244.0/22;
set_real_ip_from 103.22.200.0/22;
set_real_ip_from 103.31.4.0/22;
set_real_ip_from 104.16.0.0/13;
set_real_ip_from 104.24.0.0/14;
set_real_ip_from 108.162.192.0/18;
set_real_ip_from 131.0.72.0/22;
set_real_ip_from 141.101.64.0/18;
set_real_ip_from 162.158.0.0/15;
set_real_ip_from 172.64.0.0/13;
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 188.114.96.0/20;
set_real_ip_from 190.93.240.0/20;
set_real_ip_from 197.234.240.0/22;
set_real_ip_from 198.41.128.0/17;
set_real_ip_from 2400:cb00::/32;
set_real_ip_from 2606:4700::/32;
set_real_ip_from 2803:f800::/32;

real_ip_header CF-Connecting-IP;
real_ip_recursive on;
```

13.3 Step 2 – Real IP Snippet on WordPress Nginx Server

13.3.1 Create Reusable Snippet File

```
# Create snippets directory if it doesn't exist
mkdir -p /etc/nginx/snippets

# Create snippet with Cloudflare IPs + NPM
cat > /etc/nginx/snippets/real-ip-cloudflare.conf << 'SNIPEOF'
# Auto-generated - update monthly with the script from section 13.6
# Cloudflare IPv4 IPs
set_real_ip_from 103.21.244.0/22;
set_real_ip_from 103.22.200.0/22;
set_real_ip_from 103.31.4.0/22;
set_real_ip_from 104.16.0.0/13;
set_real_ip_from 104.24.0.0/14;
set_real_ip_from 108.162.192.0/18;
set_real_ip_from 131.0.72.0/22;
set_real_ip_from 141.101.64.0/18;
set_real_ip_from 162.158.0.0/15;
set_real_ip_from 172.64.0.0/13;
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 188.114.96.0/20;
set_real_ip_from 190.93.240.0/20;
set_real_ip_from 197.234.240.0/22;
set_real_ip_from 198.41.128.0/17;
# Cloudflare IPv6 IPs
set_real_ip_from 2400:cb00::/32;
set_real_ip_from 2606:4700::/32;
set_real_ip_from 2803:f800::/32;
set_real_ip_from 2405:b500::/32;
set_real_ip_from 2405:8100::/32;
set_real_ip_from 2a06:98c0::/29;
set_real_ip_from 2c0f:f248::/32;
# Internal Nginx Proxy Manager IP (replace with actual IP)
set_real_ip_from 192.168.1.100;
set_real_ip_from 172.16.0.0/12;
set_real_ip_from 127.0.0.1;
# Header from which we read the real IP forwarded by NPM
real_ip_header X-Real-IP;
real_ip_recursive on;
SNIPEOF

chmod 644 /etc/nginx/snippets/real-ip-cloudflare.conf
```

13.3.2 Update Virtual Host site1.conf

Add the following directives to the HTTPS server block (after the listen line):

```
server {
    listen 443 ssl http2;
    server_name wp01.ernu.sec;
    root /var/www/site1;

    # INCLUDE REAL IP CLOUDFLARE VIA NPM
    include /etc/nginx/snippets/real-ip-cloudflare.conf;

    # EXTENDED LOG FORMAT WITH CLOUDFLARE DATA
    # $remote_addr = real visitor IP (after real_ip_header)
    log_format cf_real '$remote_addr - $remote_user [$time_local]'
        '"$request" $status $body_bytes_sent'
        '"$http_referer" "$http_user_agent"'
        'cf=$http_cf_connecting_ip'
        'country=$http_cf_ipcountry'
        'ray=$http_cf_ray'
        'rt=$request_time';
```

```

access_log /var/log/nginx/site1-access.log cf_real;
error_log /var/log/nginx/site1-error.log warn;

# ... rest of configuration from Chapter 4 remains identical
}

# Test configuration
nginx -t
# Apply
systemctl reload nginx

# Verify that $remote_addr is the real visitor IP
tail -f /var/log/nginx/site1-access.log
# Correct: 85.204.x.x - - [07/Jun/2026:10:23:11 +0300] ...
# Wrong: 192.168.1.x - - [07/Jun/2026:10:23:11 +0300] ...
✓ $remote_addr in logs = the real visitor IP, not the NPM or Cloudflare IP

```

13.4 Step 3 – Restrict Direct Access (Bypass Cloudflare)

The WordPress Nginx server must accept connections ONLY from Nginx Proxy Manager. Otherwise, someone can bypass Cloudflare WAF by accessing the server IP directly.

```

# In the HTTPS server block - add access restriction:
server {
    # ...
    # ALLOW ACCESS ONLY FROM NPM
    # Replace with the actual NPM server IP
    allow 192.168.1.100;      # NPM IP
    allow 127.0.0.1;         # localhost
    allow 172.16.0.0/12;     # Docker bridge (if NPM is Docker)
    deny all;                # block everything else
    # ...
}

```

Alternative approach via UFW (recommended if NPM and WP are on separate servers):

```

# On the WordPress server - allow ports 80/443 ONLY from NPM
# Delete existing public rules
ufw delete allow 80/tcp
ufw delete allow 443/tcp

# Allow only from NPM IP
ufw allow from 192.168.1.100 to any port 80 comment 'NPM->WP HTTP'
ufw allow from 192.168.1.100 to any port 443 comment 'NPM->WP HTTPS'
ufw reload && ufw status verbose

```

! Replace 192.168.1.100 with the actual IP of your Nginx Proxy Manager server!

13.5 Step 4 – Verify Fail2Ban with Real IP

With the configuration above, Fail2Ban automatically reads the real IP from logs because \$remote_addr = real IP. Verify the jail reads the correct log:

```

# Verify logpath in jail.d/wordpress.local is correct
grep logpath /etc/fail2ban/jail.d/wordpress.local
# Should show: logpath = /var/log/nginx/site1-access.log

# Test IP parsing from log
fail2ban-regex /var/log/nginx/site1-access.log \
/etc/fail2ban/filter.d/wordpress-auth.conf
# Should find matches with real public IPs

# Verify currently banned IPs
fail2ban-client status wordpress-auth

# Manual ban/unban test
fail2ban-client set wordpress-auth banip 1.2.3.4

```

```
fail2ban-client set wordpress-auth unbanip 1.2.3.4
```

✓ Fail2Ban bans the real attacker IPs, not Cloudflare (104.x.x.x) or NPM (192.168.x.x) IPs

13.6 Step 5 – Automatic Cloudflare IP Updates

Cloudflare periodically adds new IP ranges. The following script updates them monthly:

```
cat > /usr/local/bin/update-cloudflare-ips.sh << 'CFEOF'
#!/bin/bash
SNIPPET='/etc/nginx/snippets/real-ip-cloudflare.conf'
NPM_IP='192.168.1.100'

echo "# Auto-generated: $(date)" > $SNIPPET
echo '# Cloudflare IPv4 IPs' >> $SNIPPET
curl -sf https://www.cloudflare.com/ips-v4 | while read ip; do
    echo "set_real_ip_from ${ip};" >> $SNIPPET
done
echo '# Cloudflare IPv6 IPs' >> $SNIPPET
curl -sf https://www.cloudflare.com/ips-v6 | while read ip; do
    echo "set_real_ip_from ${ip};" >> $SNIPPET
done
echo '# Nginx Proxy Manager' >> $SNIPPET
echo "set_real_ip_from ${NPM_IP};" >> $SNIPPET
echo 'set_real_ip_from 172.16.0.0/12;' >> $SNIPPET
echo 'set_real_ip_from 127.0.0.1;' >> $SNIPPET
echo 'real_ip_header X-Real-IP;' >> $SNIPPET
echo 'real_ip_recursive on;' >> $SNIPPET

nginx -t && systemctl reload nginx
echo "[$(date)] Cloudflare IPs updated OK"
CFEOF

chmod 700 /usr/local/bin/update-cloudflare-ips.sh
/usr/local/bin/update-cloudflare-ips.sh

# Add to crontab - monthly update at 03:00
(crontab -l 2>/dev/null; echo '0 3 1 * * /usr/local/bin/update-cloudflare-ips.sh >>
/var/log/cf-update.log 2>&1') | crontab -
```

13.7 Configuration Summary Table per Level

Level	Component	Key Configuration	Effect
1 – Edge	Cloudflare	Proxy ON (orange cloud)	Adds CF-Connecting-IP with real IP
2 – Proxy	NPM	proxy_set_header X-Real-IP \$http_cf_connecting_ip	Forwards real IP to Nginx WP
3 – Backend	Nginx WP	include real-ip-cloudflare.conf	\$remote_addr = real IP in logs
3 – Backend	Fail2Ban	logpath = /var/log/nginx/site1-access.log	Bans real IPs, not proxy IPs
3 – Backend	UFW	allow from NPM_IP port 80/443; deny all	Blocks Cloudflare bypass
Auto	Monthly cron	update-cloudflare-ips.sh	Cloudflare IPs always up to date

ANNEX A. GitHub Repository and Project Delivery

Official branding: Anatolie Ernu - IT & Security Solutions - <https://www.ernu.eu>

Repository: wordpress-enterprise-security-platform

A.1 Repository Structure

```
wordpress-enterprise-security-platform/
├── docs/
├── examples/
├── scripts/
├── templates/
├── .env.example
├── README.md
├── SECURITY.md
├── CHANGELOG.md
└── LICENSE
```

A.2 GitHub Import Commands

```
cd /opt/wordpress-enterprise-security-platform
git init
git add .
git commit -m "Initial release v1.0"
git branch -M main
git remote add origin https://github.com/anatolie-ernu/wordpress-enterprise-security-platform.git
git push -u origin main
git tag -a v1.0 -m "wordpress-enterprise-security-platform v1.0"
git push origin v1.0
```

A.3 Mandatory Post-Installation Checks

Component	Command	Expected Result
System	cat /etc/os-release	Debian 13
Firewall	ufw status verbose	Default deny incoming
Services	systemctl --failed	0 failed units
Nginx	nginx -t	syntax is ok
PHP-FPM	systemctl status php8.3-fpm	active/running
MariaDB	mysqladmin ping	mysql is alive

A.4 Password and Secrets Policy

⚠ The real .env file is NEVER uploaded to GitHub. Only .env.example with placeholder values is published.

```
openssl rand -base64 32
openssl rand -hex 32
```

A.5 Standard Domains and Hostnames

- Public branding: <https://www.ernu.eu>
- Technical documentation domain: ernu.sec
- Administrative email example: admin@ernu.sec

ANNEX B. Extended Fail2Ban + Nginx WordPress Protection

This annex integrates useful components from the supplementary wordpress-nginx-fail2ban-hardening archive and adapts them to the wordpress-enterprise-security-platform standard.

B.1 Added Components

```
templates/fail2ban/filter.d/wordpress-login.conf
templates/fail2ban/filter.d/wordpress-scan.conf
templates/fail2ban/filter.d/nginx-404.conf
templates/fail2ban/jail.d/wordpress-enterprise-extra.local
templates/nginx/wordpress-security-include.conf
scripts/install-wordpress-nginx-fail2ban.sh
checklists/security-checklist.md
```

B.2 Quick Installation

```
cd /opt/wordpress-enterprise-security-platform
sudo bash scripts/install-wordpress-nginx-fail2ban.sh /var/log/nginx/access.log
fail2ban-client status wordpress-login
fail2ban-client status wordpress-scan
fail2ban-client status nginx-404
```

B.3 Fail2Ban Filters

```
[wordpress-login]
failregex = ^<HOST> - .* "(GET|POST).*wp-login\.php.*" (200|302|401|403).*

[wordpress-scan]
failregex = ^<HOST> - .* "(GET|POST|HEAD).*(wp-config\.php|\.env|\.git|xmlrpc\.php|
phpmyadmin|adminer|wlwmanifest\.xml|wp-content/debug\.log).*" .*

[nginx-404]
failregex = ^<HOST> - .* "(GET|POST|HEAD) .*" 404 .*
```

B.4 Nginx Security Include

```
location = /xmlrpc.php {
    deny all;
    access_log off;
    log_not_found off;
    return 403;
}

location ~* /(\.env|\.git|wp-config\.php|readme\.html|license\.txt|composer\.(json|lock)|
package(-lock)?\.json) {
    deny all;
    access_log off;
    log_not_found off;
    return 403;
}

location ~* /wp-content/uploads/.*\.php$ {
    deny all;
    access_log off;
    log_not_found off;
    return 403;
}
```

B.5 Operational Checklist

- ☐ Real IP in Nginx logs is validated before banning.
- ☐ wp-login.php is monitored by Fail2Ban.

- ☐ Scans targeting .env, .git, wp-config.php and phpmyadmin are detected.
- ☐ xmlrpc.php is blocked if not used.
- ☐ PHP execution in uploads is blocked.
- ☐ fail2ban-client status shows active jails.

ANNEX C. Architecture Planning, Names, IP Addresses and Ports

This annex explicitly establishes how everything works when Nginx Proxy Manager, Nginx backend, WordPress, PHP-FPM, and MariaDB all run on the same Linux server. Two services cannot simultaneously listen publicly on 0.0.0.0:80 or 0.0.0.0:443 on the same IP. Therefore, Nginx Proxy Manager remains the only public-facing service on 80/443, while the WordPress Nginx backend runs locally only on internal ports.

Base rule: Nginx Proxy Manager listens publicly on 80/443; WordPress sites listen locally on 127.0.0.1:8081, 127.0.0.1:8082, etc.; SSL certificates are generated in Nginx Proxy Manager via Cloudflare DNS Challenge.

C.1 Server, Services, and Addresses Planning Table

Component	Hostname / Name	Address / IP	Ports	Role
Linux Host	srv-wp01.ernu.sec	10.10.10.20 / Public IP NAT	22/2222 SSH; 80/443 via NPM	Debian 13 hosting all services
NPM	proxy01.ernu.sec	10.10.10.20	80/443 public; :81 LAN admin	Public reverse proxy, TLS
MariaDB	db01	127.0.0.1	3306 local/Docker	Shared instance, separate DBs
WP DB site1	wp01_db	MariaDB db01 internal	DB	Database for wp01.ernu.sec
WP DB site2	wp02_db	MariaDB db01 internal	DB	Database for wp02.ernu.sec
NPM DB	npm	MariaDB db01 internal	DB	Separate DB for NPM
Nginx site1	wp01.ernu.sec	127.0.0.1	8081/TCP local	WP backend site1, no local TLS
Nginx site2	wp02.ernu.sec	127.0.0.1	8082/TCP local	WP backend site2, no local TLS
PHP-FPM site1	php8.3-wp01.sock	Unix socket	Socket	PHP execution for site1
PHP-FPM site2	php8.3-wp02.sock	Unix socket	Socket	PHP execution for site2

C.3 Logical Traffic Flow

```

Internet / Cloudflare
|
v
Nginx Proxy Manager on Linux server
- 10.10.10.20:80
- 10.10.10.20:443
|
+--> wp01.ernu.sec -> http://127.0.0.1:8081 -> Nginx backend -> PHP-FPM wp01 -> MariaDB
wp01_db
|
+--> wp02.ernu.sec -> http://127.0.0.1:8082 -> Nginx backend -> PHP-FPM wp02 -> MariaDB
wp02_db

```

ANNEX D. Shared MariaDB for WordPress and NPM

For operational simplicity, the same MariaDB instance can be used for WordPress sites and Nginx Proxy Manager on the same server. However, the same database/schema is never shared. Separate databases and users are created so that compromising one application does not grant direct access to the other's data.

D.1 Recommended Databases and Users

Application	Database	DB User	Allowed Host	Notes
WordPress site1	wp01_db	wp01_user	localhost	Used by /var/www/wp01
WordPress site2	wp02_db	wp02_user	localhost	Used by /var/www/wp02
NPM	npm	npm_user	172.30.10.%	Access from Docker npm_net

D.2 MariaDB Configuration for Local and Docker Subnet Access

If NPM runs in a Docker container and MariaDB runs on the Linux host, the container cannot use 127.0.0.1 to reach MariaDB on the host. Therefore, host.docker.internal mapped to host-gateway is used along with a Docker network with a fixed subnet.

```
# Recommended file:
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

```
[mysqld]
bind-address = 0.0.0.0
skip-networking = 0
```

```
# Restart MariaDB after modification
systemctl restart mariadb
systemctl status mariadb
```

i Opening bind-address to 0.0.0.0 must be compensated with firewall rules and MariaDB users limited by host/subnet. Never create DB users with global access without strong passwords.

D.3 Create Databases for WordPress and NPM

```
mysql -u root -p
```

```
CREATE DATABASE wp01_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp01_user'@'localhost' IDENTIFIED BY 'COMPLEX_PASSWORD_WP01';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
ON wp01_db.* TO 'wp01_user'@'localhost';
```

```
CREATE DATABASE wp02_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp02_user'@'localhost' IDENTIFIED BY 'COMPLEX_PASSWORD_WP02';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER
ON wp02_db.* TO 'wp02_user'@'localhost';
```

```
CREATE DATABASE npm CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'npm_user'@'172.30.10.%' IDENTIFIED BY 'COMPLEX_PASSWORD_NPM';
GRANT ALL PRIVILEGES ON npm.* TO 'npm_user'@'172.30.10.%';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'npm_user'@'172.30.10.%';
EXIT;
```

D.4 MariaDB Firewall Rules

```
# Allow MariaDB access only from Docker subnet used by NPM
ufw allow from 172.30.10.0/24 to any port 3306 proto tcp comment 'MariaDB for NPM docker network'
```

```
# Verify
ufw status verbose
ss -lntp | grep 3306
```

ANNEX E. docker-compose.yml for NPM with Shared MariaDB

This docker-compose starts only the Nginx Proxy Manager container. The database is no longer a separate npm-db container but the same MariaDB instance used by WordPress sites, with a separate npm database and separate npm_user user.

E.1 Directory Structure

```
mkdir -p /opt/npm
cd /opt/npm
mkdir -p data letsencrypt logs
chmod 750 data letsencrypt logs
```

E.2 .env File

```
cat > /opt/npm/.env << 'EOF'
TZ=Europe/Chisinau
NPM_DB_HOST=host.docker.internal
NPM_DB_PORT=3306
NPM_DB_NAME=npmm
NPM_DB_USER=npmm_user
NPM_DB_PASSWORD=COMPLEX_PASSWORD_NPM
EOF
chmod 600 /opt/npm/.env
```

E.3 Final docker-compose.yml

```
cat > /opt/npm/docker-compose.yml << 'EOF'
services:
  npm-app:
    image: jc21/nginx-proxy-manager:latest
    container_name: npm-app
    restart: unless-stopped
    ports:
      - '80:80'
      - '443:443'
      - '10.10.10.20:81:81'
    environment:
      TZ: ${TZ}
      DB_MYSQL_HOST: ${NPM_DB_HOST}
      DB_MYSQL_PORT: ${NPM_DB_PORT}
      DB_MYSQL_USER: ${NPM_DB_USER}
      DB_MYSQL_PASSWORD: ${NPM_DB_PASSWORD}
      DB_MYSQL_NAME: ${NPM_DB_NAME}
    volumes:
      - ./data:/data
      - ./letsencrypt:/etc/letsencrypt
      - ./logs:/data/logs
    extra_hosts:
      - 'host.docker.internal:host-gateway'
    networks:
      - npm_net

networks:
  npm_net:
    name: npm_net
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.10.0/24
EOF
```

E.4 Start and Verify NPM

```
cd /opt/npm
```

```
docker compose config
docker compose pull
docker compose up -d
docker compose ps
docker logs -f npm-app
```

```
# Verify public ports and local admin
ss -lntp | egrep '':80|:443|:81'
```

```
# Verify externally mapped logs
ls -la /opt/npm/logs
```

ANNEX F. SSL Certificates in NPM via Cloudflare DNS Challenge

SSL certificates for WordPress domains are generated in Nginx Proxy Manager, not in the Nginx backend. The Nginx backend remains HTTP local on 127.0.0.1:8081/8082. DNS Challenge with a Cloudflare token at the DNS zone level is used for certificate issuance.

F.1 Create API Token in Cloudflare

In Cloudflare, create an API Token dedicated exclusively to DNS Challenge. The token must have minimal permissions, not Global API Key.

Setting	Recommended Value
Permissions	Zone - DNS - Edit
Zone Resources	Include - Specific zone - ernu.sec / actual domain
Client IP Filtering	Optional: public IP of NPM server
TTL token	Optional, per internal policy

F.2 Enter Certificate in NPM

In the Nginx Proxy Manager interface: SSL Certificates → Add SSL Certificate → Let's Encrypt. Enter domains, email address, and enable the Use a DNS Challenge option. The selected provider is Cloudflare.

```
dns_cloudflare_api_token = CLOUDFLARE_DNS_CHALLENGE_TOKEN
```

i The Cloudflare token is not entered in `docker-compose.yml` and is not published to GitHub. It is entered only in the NPM interface or through a secure secrets mechanism.

ANNEX G. Local WordPress Backend Installation and NPM Mapping

This annex completes the WordPress installation for the model with Nginx Proxy Manager in front. WordPress does not directly expose HTTPS. The local backend listens on HTTP at 127.0.0.1, and TLS is terminated in NPM.

G.1 Create System User and Directories

```
useradd -r -s /usr/sbin/nologin -d /var/www/wp01 wp01
mkdir -p /var/www/wp01
chown -R wp01:wp01 /var/www/wp01
```

G.2 Download WordPress

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
rsync -av wordpress/ /var/www/wp01/
chown -R wp01:wp01 /var/www/wp01
find /var/www/wp01 -type d -exec chmod 755 {} \;
find /var/www/wp01 -type f -exec chmod 644 {} \;
```

G.3 Configure wp-config.php

```
cp /var/www/wp01/wp-config-sample.php /var/www/wp01/wp-config.php
nano /var/www/wp01/wp-config.php
```

```
# Main settings
define('DB_NAME', 'wp01_db');
define('DB_USER', 'wp01_user');
define('DB_PASSWORD', 'COMPLEX_PASSWORD_WP01');
define('DB_HOST', 'localhost');
define('DB_CHARSET', 'utf8mb4');
define('DB_COLLATE', '');

# Hardening recommendations
define('DISALLOW_FILE_EDIT', true);
define('FORCE_SSL_ADMIN', true);
define('FS_METHOD', 'direct');
```

G.4 Nginx Backend Site 1 on Local Port 8081

```
cat > /etc/nginx/sites-available/wp01-backend.conf << 'EOF'
server {
    listen 127.0.0.1:8081;
    server_name wp01.ernu.sec;
    root /var/www/wp01;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ /\.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php/php8.3-wp01.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }

    location = /xmlrpc.php { deny all; }
    location ~* /wp-content/uploads/.*\.php$ { deny all; }
    location ~ /\. { deny all; }
}
```

EOF

```
ln -sf /etc/nginx/sites-available/wp01-backend.conf /etc/nginx/sites-enabled/  
nginx -t  
systemctl reload nginx
```

G.5 Final Verifications

```
# NPM should listen publicly on 80 and 443  
ss -lntp | egrep ':\:80|:\:443'
```

```
# NPM admin interface should only be accessible from LAN/VPN  
ss -lntp | grep ':\:81'
```

```
# WordPress backend should be local only  
ss -lntp | grep ':\:8081'
```

```
# Local backend test  
curl -I http://127.0.0.1:8081
```

```
# Test via domain after NPM and Cloudflare configuration  
curl -I https://wp01.ernu.sec
```

ANNEX H. NPM Management Access from Local Network

The Nginx Proxy Manager management interface must not be published to the Internet but must be accessible to administrators from the local network. For this reason, port 81 is not mapped to localhost but to the server's LAN address, and access is restricted via firewall to the administrative subnet only.

Recommended model: NPM admin on 10.10.10.20:81, access allowed only from LAN/VPN, blocked from Internet.

H.1 Correct Mapping in docker-compose.yml

```
ports:
  - '80:80'
  - '443:443'
  - '10.10.10.20:81:81'
```

H.2 Firewall Rules for LAN Access

```
# Allow access to NPM interface only from local admin network
ufw allow from 10.10.10.0/24 to any port 81 proto tcp comment 'NPM admin LAN only'

# Optional: if server has a public interface, explicitly block public access to port 81
ufw deny 81/tcp comment 'Block public NPM admin'

# Verify rules
ufw status numbered
ss -lntp | grep ':81'
```

H.3 Administration URL

```
http://10.10.10.20:81
# Access allowed only from LAN/VPN, not from the Internet.
```